

阿里巴巴前端开发工程师5年面经

阿里巴巴前端开发工程师面经

一面

1. 介绍项目
2. 事件委托机制
3. 深拷贝和浅拷贝
4. 快排算法
5. 前端跨域
6. webpack工作原理
7. CSRF攻击及防范
8. XSS攻击及防范
9. 正则表达式
10. splice参数问题

二面

1. 项目深挖
2. bind, call, apply用法及区别
3. 数组的原生方法
4. slice会改变原始数组吗
5. vue2.0和vue3.0的区别
6. 前后端联合开发所遇到的问题
7. 微信小程序watch监听实现方式与场景
8. 技术上解决最难的问题

参考答案：

2. 事件委托机制：

- 定义：事件委托是一种利用事件冒泡机制，将事件处理程序绑定到父元素上，从而管理多个子元素的事件。通过在父元素上绑定事件处理程序，可以减少内存占用，提高性能。
- 实现：

```
1 <ul id="parent">
2   <li>Item 1</li>
3   <li>Item 2</li>
4   <li>Item 3</li>
5 </ul>
6
7 <script>
8   document.getElementById('parent').addEventListener('click',
9     function(event) {
10       if (event.target && event.target.nodeName === 'LI') {
11         console.log('List item ', event.target.textContent, ' was clicked.');
```

3. 深拷贝和浅拷贝：

- 浅拷贝：拷贝对象的引用，即两个对象共享同一块内存。修改一个对象会影响另一个对象。

```
1 let obj1 = { a: 1, b: { c: 2 } };
2 let obj2 = Object.assign({}, obj1);
3 obj2.b.c = 3;
4 console.log(obj1.b.c); // 3
```

- 深拷贝：拷贝对象的所有属性及嵌套对象，两个对象互不影响。

```
1 let obj1 = { a: 1, b: { c: 2 } };
2 let obj2 = JSON.parse(JSON.stringify(obj1));
3 obj2.b.c = 3;
4 console.log(obj1.b.c); // 2
```

4. 快排算法：

- 定义：快速排序是一种分治算法，通过递归将数组分为较小和较大的子数组，然后排序子数组。
- 实现：

```

1 function quickSort(arr) {
2   if (arr.length <= 1) return arr;
3   let pivot = arr[Math.floor(arr.length / 2)];
4   let left = arr.filter(x => x < pivot);
5   let right = arr.filter(x => x > pivot);
6   return [...quickSort(left), pivot, ...quickSort(right)];
7 }
8
9 let array = [3, 6, 8, 10, 1, 2, 1];
10 console.log(quickSort(array)); // [1, 1, 2, 3, 6, 8, 10]

```

5. 前端跨域：

- CORS（跨域资源共享）：通过设置服务器响应头 `Access-Control-Allow-Origin` 来允许特定域名访问资源。

```

1 app.use((req, res, next) => {
2   res.header('Access-Control-Allow-Origin', '*');
3   res.header('Access-Control-Allow-Methods', 'GET, POST, PUT, DELETE');
4   res.header('Access-Control-Allow-Headers', 'Content-Type, Authorization');
5   next();
6 });

```

6. webpack工作原理：

- 定义：webpack 是一个模块打包工具，它将多个模块打包成一个或多个 bundle 文件。
- 工作流程：
 - 入口：确定从哪个模块开始。
 - 模块解析：递归解析依赖树，加载模块。
 - 插件：在构建过程中使用插件处理额外任务。
 - 输出：生成最终的打包文件。
 - 示例配置：

```

1 const path = require('path');
2 module.exports = {
3   entry: './src/index.js',
4   output: {
5     filename: 'bundle.js',
6     path: path.resolve(__dirname, 'dist')
7   },
8   module: {

```

```
9     rules: [
10       { test: /\.css$/, use: ['style-loader', 'css-loader'] }
11     ],
12   },
13   plugins: [
14     new HtmlWebpackPlugin({ template: './src/index.html' })
15   ]
16 };
```

7. CSRF攻击及防范：

- 定义：跨站请求伪造（CSRF）是一种攻击，攻击者诱导用户在已认证的会话中执行不想要的操作。
- 防范措施：
 - 使用 CSRF token：在表单或请求中加入唯一的 token，并在服务器端验证。
 - 检查 Referer 头：验证请求的来源是否可信。
 - 示例：

```
1 // 生成 token
2 const csrfToken = generateCSRFToken();
3 // 在表单中加入 token
4 <input type="hidden" name="_csrf" value="<%= csrfToken %>">
```

8. XSS攻击及防范：

- 定义：跨站脚本攻击（XSS）是指通过注入恶意脚本代码，使其在用户浏览器上执行。
- 防范措施：
 - 输入验证和过滤：对所有用户输入进行严格的验证和过滤。
 - 输出编码：对输出内容进行编码，防止脚本执行。
 - 使用 CSP：设置内容安全策略，限制可执行脚本的来源。
 - 示例：

```
1 const sanitizeHtml = require('sanitize-html');
2 const clean = sanitizeHtml(dirty);
```

9. 正则表达式：

- 定义：正则表达式是一种用于匹配字符串的模式。

- 基础语法：

- 字符类：`[abc]` 匹配 `a`、`b` 或 `c`
- 量词：`a*` 匹配零个或多个 `a`
- 分组：`(abc)` 匹配 `abc`
- 锚点：`^` 表示开始，`$` 表示结束
- 示例：

```
1 const regex = /\d{3}-\d{2}-\d{4}/;  
2 console.log(regex.test('123-45-6789')); // true
```

10. splice参数问题：

- 定义：`splice` 方法通过删除或添加元素来修改数组内容，并返回被删除的元素。
- 参数：
 - `start`：指定从哪个位置开始删除元素（从 0 开始）。
 - `deleteCount`：表示要删除的元素数量。
 - `items`：要添加到数组中的新元素（可选）。
- 示例：

```
1 let array = [1, 2, 3, 4, 5];  
2 let removed = array.splice(1, 2, 'a', 'b');  
3 console.log(array); // [1, 'a', 'b', 4, 5]  
4 console.log(removed); // [2, 3]
```

二面参考答案

针对您提出的问题，我将逐一进行详细解答：

2. bind, call, apply用法及区别

- `bind`：`bind` 方法创建一个新的函数，当这个新函数被调用时，`bind` 的第一个参数将作为它运行时的 `this`，其余参数将作为新函数运行时的初始参数，供调用时使用。`bind` 不会立即执行函数，而是返回一个新的函数。
- `call`：`call` 方法使用一个指定的 `this` 值和一个提供的参数列表（以逗号分隔）调用一个函数。它会立即执行函数。

- `apply`: `apply` 方法调用一个具有给定 `this` 值的函数，以及以一个数组（或类数组对象）的形式提供的参数。与 `call` 不同，`apply` 接受的是一个参数数组，而不是多个参数。它也会立即执行函数。
- 区别: `bind` 是返回一个新的函数，不会立即执行；而 `call` 和 `apply` 都会立即执行函数。另外，`call` 需要显式地传递参数，而 `apply` 则以数组形式传递参数。

3. 数组的原生方法

- 数组和字符串的转换方法: `toString()`、`join()`。
- 数组尾部操作方法: `push()`、`pop()`。
- 数组首部操作方法: `shift()`、`unshift()`。
- 排序方法: `sort()`。
- 拼接方法: `concat()`。
- 截取方法: `slice()`。
- 插入/删除方法: `splice()`。
- 查找方法: `indexOf()`、`lastIndexOf()`。
- 迭代方法: `every()`、`some()`、`filter()`、`forEach()`、`reduce()`、`reduceRight()`。

4. slice会改变原始数组吗

- `slice` 方法不会改变原始数组。它返回一个新的数组，该数组包含从原始数组中选定的元素（从起始索引到结束索引，但不包括结束索引处的元素）。

5. vue2.0和vue3.0的区别

- 性能: Vue 3重写了虚拟DOM的实现，并优化了编译模板，使得性能有了显著提升。
- 体积: Vue 3通过webpack的tree-shaking功能，能够仅打包需要的模块，从而减小了打包后的体积。
- 维护性: Vue 3引入了Composition API，可以与现有的Options API一起使用，使得逻辑组合与复用更加灵活。
- Typescript支持: Vue 3是基于Typescript编写的，因此能够享受到自动的类型定义提示。
- 原生支持: Vue 3允许自定义渲染API，使得其更接近原生。
- 响应式API: Vue 3将响应式API暴露出来，使得开发者可以更加灵活地控制响应式数据。

6. 前后端联合开发所遇到的问题

- API接口返回数据格式问题: 如返回的是JSON字符串而非JSON对象，会导致前端无法正确解析数据。
- 编码问题: 如MySQL数据表的编码错误，可能导致接口报错。

- 环境差异问题：生产环境和本地环境可能存在差异，如数据库表格字段、property文件配置参数等不一致，导致报错。
- 文件大小限制问题：如后端Nginx限制上传文件大小，可能导致上传大文件失败。
- 请求超时问题：如Nginx设置超时时间太短，可能导致前端请求一段时间后接口报错。

7. 微信小程序watch监听实现方式与场景

- 在微信小程序中，可以通过封装 `watch` 监听方法来实现类似Vue的数据监听功能。这通常涉及在页面或组件的生命周期中注册监听器，并在数据变化时触发相应的回调函数。
- 场景包括但不限于：监听用户输入、监听页面状态变化、监听全局状态变化等。通过监听这些数据变化，开发者可以在数据变化时执行相应的逻辑，如更新页面UI、发送网络请求等。

8. 技术上解决最难的问题

- 技术上解决最难的问题通常涉及多个方面，包括深入理解问题的本质、排查并定位问题的根源、选择合适的解决方案以及确保解决方案的稳定性和可靠性。
- 解决技术难题的思路包括：深刻理解问题的现象（如报错信息、日志信息等）、列出可能导致问题的原因并进行逐一测试、使用排除法或控制变量法缩小问题范围、深入研究相关技术的原理和文档等。
- 在实际开发中，解决技术难题还需要具备良好的编程基础、丰富的经验积累以及良好的团队协作和沟通能力。